Answer Key

Self-Study Week: 8051 Chapter 6 (instructor only, 52 marks)

Part A: ADD and ADDC

A1. *(2 marks)* Give the final values of A (in hex), CY, and AC after this code.

```
CLR   C
MOV   A, #0F5H
ADDC  A, #0BH
```

Answer: A = 00H, CY = 1, AC = 1.

Working: F5H + 0BH = 100H, so A = 00H and there is a carry out of bit 7 (CY = 1). In the low nibble, 5 + B = 10H, so there is a carry from bit 3 (AC = 1).

A2. *(2 marks)* Give the final values of A (in hex), CY, and AC after this code.

```
SETB  C
MOV   A, #3AH
ADDC  A, #26H
```

Answer: A = 61H, CY = 0, AC = 1.

Working: 3AH + 26H + 1 = 61H. In the low nibble, A + 6 + 1 = 11H, so AC = 1. There is no carry out of bit 7.

A3. *(3 marks)* Write a program that adds the 16-bit numbers 12B7H and 0E69H. Save the result in R6 (low byte) and R7 (high byte). State the final values of R7 and R6.

Answer: R7 = 21H and R6 = 20H (result 2120H).

Working: low bytes B7H + 69H = 120H, so the low byte is 20H and CY = 1. High bytes 12H + 0EH + 1 = 21H.

```
CLR   C
MOV   A, #0B7H
ADDC  A, #69H      ; A = 20H, CY = 1
MOV   R6, A
MOV   A, #12H
ADDC  A, #0EH      ; A = 12H + 0EH + 1 = 21H
MOV   R7, A
```

A4. *(1 mark)* A student runs MOV A,#7 followed by ADDC A,#2 in two different programs. One program gives A = 09H and the other gives A = 0AH. Explain why, and how to make the result always 09H.

Answer: ADDC also adds the carry flag. If CY = 0 the result is 09H; if CY = 1 (left over from an earlier instruction) the result is 0AH. Use CLR C before ADDC (or use ADD) so that the result is always 09H.

Part B: SUBB

B1. *(2 marks)* Give the final values of A (in hex), CY, and AC after this code.

```
MOV   A, #52H
CLR   C
SUBB  A, #2FH
```

Answer: A = 23H, CY = 0, AC = 1.

Working: 52H - 2FH = 23H. The result is positive, so CY = 0. The low nibble 2 is smaller than F, so there is a borrow from bit 4 (AC = 1).

B2. *(3 marks)* Give the final value of A and CY after this code. Is the result positive or negative? Find the size of the result.

```
MOV   A, #20H
CLR   C
SUBB  A, #30H
```

Answer: A = F0H, CY = 1. The result is negative.

Size: take the 2's complement of F0H, which is 10H, so the result is -10H (-16 in decimal).

B3. *(1 mark)* Give the final value of A after this code.

```
SETB  C
MOV   A, #09H
SUBB  A, #04H
```

Answer: A = 04H (09H - 04H - 1). The carry is subtracted too.

B4. *(1 mark)* List the three steps the CPU performs to execute SUBB.

Answer: (1) Take the 2's complement of the source. (2) Add it to A. (3) Invert CY and AC.

Part C: Packed BCD, unpacked BCD, and DA A

C1. *(2 marks)* Write the decimal number 47 as packed BCD and as unpacked BCD (in hex).

Answer: Packed BCD: 47H (0100 0111). Unpacked BCD: 04H and 07H (two bytes).

C2. *(2 marks)* Which of these bytes are valid packed BCD: 59H, 7AH, 90H, 1FH? Give the reason for each invalid one.

Answer: 59H and 90H are valid. 7AH is invalid because A is not a decimal digit, and 1FH is invalid because F is not a decimal digit (nibble codes AH to FH are not BCD digits).

C3. *(2 marks)* Give the value of A after ADD and after DA A.

```
MOV   A, #38H
ADD   A, #45H
DA    A
```

Answer: After ADD: 7DH. After DA A: 83H, CY = 0.

Reason: the low nibble D is greater than 9, so DA A adds 06H (7DH + 06H = 83H). Decimal check: 38 + 45 = 83.

C4. *(2 marks)* Give the value of A and AC after ADD, and the value of A after DA A. Explain why DA A changes the result even though the low nibble looks valid.

```
MOV   A, #38H
ADD   A, #29H
DA    A
```

Answer: After ADD: A = 61H, AC = 1. After DA A: A = 67H, CY = 0.

Reason: 8 + 9 = 11H produced a carry from bit 3 (AC = 1), so DA A adds 06H. Decimal check: 38 + 29 = 67.

C5. *(2 marks)* Give the value of A after ADD and the final values of A and CY after DA A.

```
MOV   A, #86H
ADD   A, #47H
DA    A
```

Answer: After ADD: A = CDH. After DA A: A = 33H, CY = 1 (decimal sum 133).

Working: the low nibble D > 9, so add 06H: D3H. The upper nibble D > 9, so add 60H: 133H, giving A = 33H and CY = 1.

C6. *(2 marks)* Give the value of A after ADD, and the final A and CY after DA A. Explain why DA A also adds 60H even though the upper nibble of the ADD result is 9.

```
MOV   A, #49H
ADD   A, #51H
DA    A
```

Answer: After ADD: A = 9AH. After DA A: A = 00H, CY = 1 (decimal sum 100).

Working: 9AH has low nibble A > 9, so add 06H: A0H. The upper nibble is now A > 9, so DA A adds 60H: 100H, giving A = 00H and CY = 1. The upper-nibble check is done after the low-nibble correction, so it can be triggered by that correction.

C7. *(1 mark)* True or false: DA A can be used after SUBB to correct a packed BCD subtraction. Give the reason.

Answer: False. DA A works correctly only after ADD or ADDC (and only for packed BCD).

C8. *(3 marks)* Write a program that adds the packed BCD numbers 76 and 58. Save the tens and ones digits in R6 and the hundreds digit in R7. State the final values of R7 and R6.

Answer: R7 = 01H and R6 = 34H (decimal sum 134).

Working: 76H + 58H = CEH. DA A adds 06H (D4H), then 60H (134H), giving A = 34H and CY = 1. Adding 00H to A = 00H with ADDC copies CY into A.

```
MOV   A, #76H
ADD   A, #58H
DA    A            ; A = 34H, CY = 1
MOV   R6, A
CLR   A             ; CLR A does not change CY
ADDC  A, #00H      ; A = 00H + 00H + CY = 01H
MOV   R7, A
```

Part D: MUL and DIV

D1. *(2 marks)* Give the final values of A, B, and OV (in hex where needed).

```
MOV   A, #0F0H
MOV   B, #10H
MUL   AB
```

Answer: A = 00H, B = 0FH, OV = 1.

Working: F0H x 10H = 0F00H. The high byte 0FH goes to B and the low byte 00H goes to A. B is not zero, so OV = 1.

D2. *(1 mark)* Give the final values of A, B, and OV.

```
MOV   A, #12H
MOV   B, #05H
MUL   AB
```

Answer: A = 5AH, B = 00H, OV = 0 (18 x 5 = 90 = 5AH).

D3. *(2 marks)* Give the final values of A and B in hex.

```
MOV   A, #200
MOV   B, #7
DIV   AB
```

Answer: A = 1CH (quotient 28), B = 04H (remainder 4). 200 / 7 = 28 remainder 4.

D4. *(1 mark)* What does OV = 1 after DIV AB tell you?

Answer: B was 0 (division by zero). The results in A and B are undefined.

D5. *(2 marks)* Write a program that uses DIV to test the number in R1: put 00H in R0 if the number is even and 01H if it is odd.

Answer: The remainder of dividing by 2 is 0 for an even number and 1 for an odd number, so the remainder can be copied directly. A version with JZ and two branches is also accepted.

```
MOV   A, R1
MOV   B, #2
DIV   AB          ; B = remainder (0 or 1)
MOV   R0, B
```

D6. *(2 marks)* R3 contains 9BH. Give the values of R5, R6, and R7 after this program (hex to decimal digits).

```
MOV   A, R3
MOV   B, #10
DIV   AB
MOV   R5, B
MOV   B, #10
DIV   AB
MOV   R6, B
MOV   R7, A
```

Answer: R5 = 05H, R6 = 05H, R7 = 01H.

Working: 9BH = 155. 155 / 10 = 15 remainder 5 (R5 = 5). 15 / 10 = 1 remainder 5 (R6 = 5, R7 = 1).

D7. *(2 marks)* R2 = 07H (tens digit) and R3 = 04H (ones digit). Give the final value of A in hex, and the decimal number it represents.

```
MOV   A, R2
MOV   B, #10
MUL   AB
ADD   A, R3
```

Answer: A = 4AH, which is decimal 74.

Working: 7 x 10 = 70 = 46H, and 46H + 04H = 4AH.

Part E: ANL and ORL

E1. *(1 mark)* Give the final value of A.

```
MOV   A, #0B7H
ANL   A, #0F0H
```

Answer: A = B0H (the low nibble is cleared).

E2. *(1 mark)* Write one instruction that clears bits 3 and 2 of A and leaves all other bits unchanged.

Answer: ANL A, #11110011B (or ANL A, #0F3H). The mask has 0 in bits 3 and 2 and 1 everywhere else.

E3. *(1 mark)* Write one instruction that sets bits 7 and 0 of A and leaves all other bits unchanged.

Answer: ORL A, #10000001B (or ORL A, #81H).

E4. *(2 marks)* (a) A contains 37H, the ASCII code of the character "7". Write one instruction that converts it to unpacked BCD. (b) A contains 09H (unpacked BCD). Write one instruction that converts it to ASCII.

Answer: (a) ANL A, #0FH gives A = 07H. (b) ORL A, #30H gives A = 39H.

Part F: Integrated task

F1. *(4 marks)* R2 = 34H and R3 = 36H are the ASCII codes of the digits "4" and "6". Write a program that converts them into one binary number in A (the number 46 decimal). Use ANL, MUL, and ADD. State the final value of A in hex.

Answer: A = 2EH (46 decimal).

Working: convert each ASCII code to a digit with ANL (04H and 06H), multiply the tens digit by 10 (40 = 28H), then add the ones digit (28H + 06H = 2EH).

```
MOV   A, R2
ANL   A, #0FH        ; A = 04H (tens digit)
MOV   B, #10
MUL   AB             ; A = 28H (40)
MOV   R4, A
MOV   A, R3
ANL   A, #0FH        ; A = 06H (ones digit)
ADD   A, R4          ; A = 2EH (46)
```

> **Marking:** the marks per question are shown with each question. Accept any correct program that gives the stated result (for example, a different register or an equivalent instruction), and give part marks for correct method with an arithmetic slip.